



Qwen3.5 多模态大语言模型转换部署到 ncnn 框架：完整技术实践



nihui
智乃是我的!

关注她

收录于 • ncnn >

100 人赞同了该文章 >

Qwen3.5+ 多模态大语言模型+转换部署到 ncnn 框架：完整技术实践

TL;DR

https://github.com/futz12/ncnn_llm
github.com/futz12/ncnn_llm

赞同 100

13 条评论

106

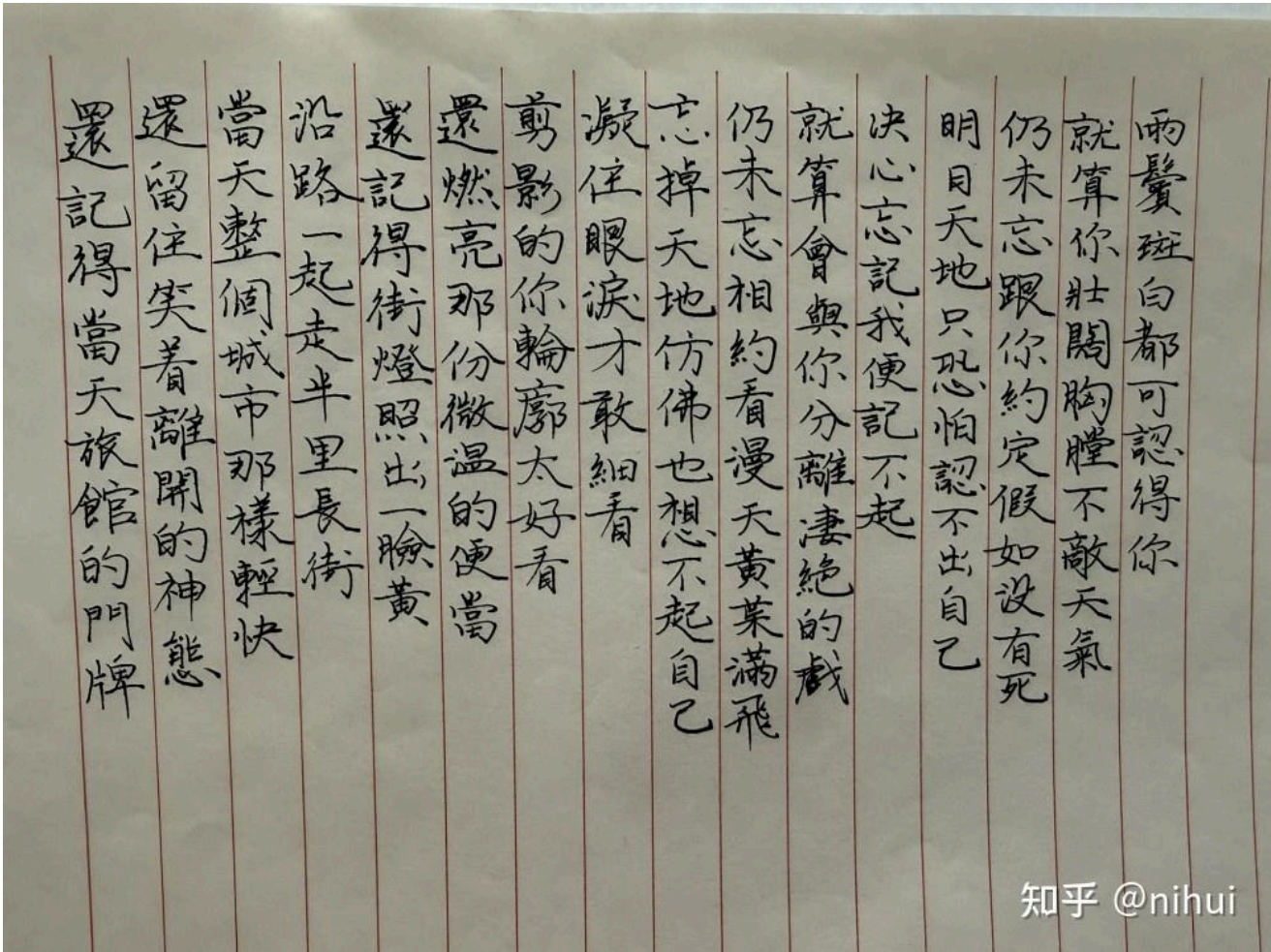
4

分享

申请转载



./llm_ncnn_run --model ./assets/qwen3.5_0.8b --image test.jpg



好的，这张图片展示了一段用黑色墨水写在带有红色横线的纸张上的中文手写字迹。

以下是图片内容的详细分析：

- **书写内容**：
这是一段写于中文的散文诗，字迹工整，行距适中，字体为行书或行楷风格。文字内容如下：

> 雨鬓斑白都可认得你
> 就算你壮阔胸膛不敵天氣
> 仍未忘跟你約定假如沒有死
> 明日天地只恐怕認不出自己
> 決心忘記我便記不起
> 就算會與你分離淒絕的戲
> 仍未忘相約看漫天黃葉滿飛
> 忘掉天地彷彿也想不起自己
> 凝住眼淚才敢細看
> 剪影的你輪廓太好看
> 還燃亮那份微溫的便當
> 還記得一起走半里長街
> 沿途一起走半里長街
> 當天整個城市那樣輕快
> 還留住笑着離開的神態
> 還記得當天旅館的門牌

- **情感基调**：
整段文字充满了浓厚的怀旧感和对逝去时光的追忆。它通过描绘一个雨中、黄昏或夜晚的场景，将“我”与一个早已逝去的恋人（或爱人）联系在一起。文字表达了对彼此之间深厚情谊的怀念，以及对那段共同经历的美好回忆的珍视。

- **画面氛围**：
虽然图片仅展示了文字，但可以想象文字所描绘的画面。文字提到“雨鬓斑白”，暗示了场景可能是在下雨或傍晚。提到了“壮阔胸膛”、“黄叶”、“夜晚”等元素，营造出一种朦胧、略带忧郁但又充满诗意的氛围。文字中“凝住眼淚”、“剪影”、“微溫的便當”、“神態”等词汇，进一步增强了画面的情感深度，仿佛读者也能听到或看到那个场景中的声音和画面。

- **文本来源**：
这段文字很可能出自某位作者的个人随笔、日记或诗歌集。其风格和内容风格与许多中国现代诗人或作家（如林庚、朱自清、郁达夫等）的作品有相似之处，可能是一首题为《雨鬓斑白》的诗歌。

- **排版**：
文字以竖排方式书写，从上到下，从右到左。在纸张的右上方，可以隐约看到“雨鬓斑白都可认得你”这几个字，这表明这段文字可能是从另一页或同一页的另一处开始，或者是作者当时正在写这首诗时，手边就写到了这里。

总而言之，这是一段充满情感与回忆的中文手写字迹，描绘了一个充满怀旧气息的雨中黄昏场景，表达了作者对逝去时光的怀念和对美好时光的眷恋。

User: []

一、引言

Qwen3.5是阿里通义千问团队推出的新一代多模态大语言模型。相较于前代 Qwen2.5-VL 和 Qwen3-VL，Qwen3.5 在架构层面进行了重大革新——引入了 **Gated DeltaNet⁺ 线性注意力机制⁺**与传统 **Full Attention⁺** 的混合架构。这种混合架构设计使模型在保持强大推理能力的同时，显著降低了**长序列推理⁺**时的内存占用和计算开销。

ncnn 是腾讯开源的高性能神经网络前向计算框架，专为移动端和嵌入式设备优化，支持 **ARM CPU⁺**、Vulkan GPU 等多种后端。将 Qwen3.5 这样的混合架构模型部署到 ncnn，意味着可以在手机、开发板等边缘设备上运行多模态大语言模型，具有重要的工程实践价值。

然而，Qwen3.5 的混合注意力架构给模型转换带来了前所未有的挑战。Gated DeltaNet 模块内含复杂的 **RNN⁺** 循环计算，直接 trace 导出会导致循

大家都在搜

换一

韩国散户杠杆爆仓 478 万

曾硬刚王健林的80后干部… 348 万

realme 手机退出中国市场 341 万

重庆彭水发生山体崩塌 320 万

人生选择模拟器 315 万

曲婉婷自曝得乳腺癌 311 万

阿根廷 2-1 英格兰 296 万

西班牙阿根廷谁能夺冠 288 万

日本护士将排泄物混入患… 276 万

邹市明夫妇2亿理财亏损始末 272 万

登录已失效

重新登录

的主过程，涵盖从模型验证、分模块导出、pnmx 中间表示修改、[自定义昇腾](#) 头现到 cache 状态注入的每一个关键步骤。

二、Qwen3.5 模型架构概览

在动手转换之前，有必要深入理解 Qwen3.5 的整体架构，这直接决定了转换策略的设计。

2.1 整体结构

Qwen3.5 作为视觉语言模型（VLM），整体由三大部分组成：

模块	功能	关键特性
Vision Encoder	将图像编码为视觉 token	基于 ViT+ ，包含 patch embedding、attention blocks、merge patch 等子模块
Token Embedding+ / LM Head+	输入 token 映射与输出 logits 生成	两者共享权重 (tied weights)
LLM Decoder+	自回归语言模型主体	混合架构：Full Attention + Gated DeltaNet 线性注意力 + Sparse MoE+

与前代 Qwen3-VL 相比，Qwen3.5 **去掉了 DeepStack Features 桥接层**，视觉编码器的输出不再需要额外的 deepstack feature tensor 做中间桥接，而是直接作为 token 序列送入 LLM Decoder。这一简化不仅减少了模型的复杂度，也为 ncnn 部署减少了一个需要单独处理的模块。

2.2 混合注意力的 Decoder 层

Qwen3.5 Decoder 的层类型由[配置文件+](#)中的 layer_types 列表指定，每一层要么是 "full_attention" 类型，要么是 "linear_attention" 类型。以 0.8B 模型为例，其 24 层 Decoder 中包含 6 个 Full Attention 层和 18 个 Linear Attention (Gated DeltaNet) 层。

Full Attention 层与经典的 [Transformer+](#) 解码器层类似，采用 Multi-Head Attention（带 GQA）配合 [RoPE+](#) 位置编码，但增加了一个创新点：Q 投影产出的维度是正常的两倍，前半部分作为 query，后半部分经过 sigmoid 门控后与注意力输出相乘。这种设计增强了模型的表达能力。Full Attention 层需要维护标准的 **KV Cache**。

Linear Attention 层 (Gated DeltaNet) 是 Qwen3.5 的核心创新。该模块不使用传统的 softmax 注意力，而是维护一个 recurrent state 矩阵，通过门控的 delta rule 进行增量更新。具体来说，输入经过[线性投影+](#)后得到 query、key、value 以及门控信号 z、beta 和 alpha，然后依次经过：

- ShortConv (短卷积+)**：对 QKV 拼接后做 depthwise causal conv1d + SiLU 激活，引入局部上下文信息
- Gated Delta Rule**：核心的线性注意力计算，维护形状为 [\[num_heads+, k_dim, v_dim\]](#) 的 recurrent state，通过门控的[衰减因子+](#)和 delta rule 进行状态更新
- Gated RMSNorm**：对输出进行归一化，使用 z 门控信号进行调制

Linear Attention 层需要维护两种 cache：**Conv State**（短卷积的历史状态）和 **GDR State**（Gated DeltaNet 的 recurrent state）。

在自回归推理中，Qwen3.5 的 Decoder 需要维护两种完全不同的 cache 状态，这是部署的核心难点：

Cache 类型	所属层	形状 (0.8B 模型)	特性
KV Cache	Full Attention 层 (6 层)	[num_heads, seq_len, head_dim+]	随序列长度+动态增长
GDR Cache	Linear Attention 层 (18 层)	[num_heads, k_dim, v_dim]	固定形状，每步原地更新
Conv Cache	Linear Attention 层 (18 层)	[conv_dim, kernel_size-1]	固定形状，滑动窗口更新

三、环境准备与模型验证

3.1 下载模型与依赖安装

首先从 [HuggingFace+](#) 下载 Qwen3.5 0.8B [模型权重+](#)。由于 Qwen3.5 使用了 Gated DeltaNet 线性注意力，[transformers+](#) 库中对应的模型类名为 Qwen3NextForCausalLM（Qwen3-Next 是 Qwen3.5 在 HuggingFace 上的正式注册名称）。

需要安装的关键依赖包括最新版 transformers（需支持 qwen3_next 模型类型）以及 torch。如果有 [CUDA+](#) 环境，还可以安装 [flash-linear-attention+](#)（fla）和 causal-conv1d 库以加速推理，但这不是必须的，transformers 中已内置了纯 [PyTorch+](#) 的 fallback 实现。

3.2 编写最小测试脚本

在开始任何转换工作之前，首先编写一个最小化测试脚本 test.py，确保模型能够在本地正确加载和推理。该脚本的核心流程是：加载 [tokenizer+](#) 和模型、构造输入、执行一轮前向推理、比较输出 logits 或生成文本是否符合预期。

这一步至关重要，它建立了一个 **正确性基线**。后续每次对模型代码做出修改（例如改写 Gated DeltaNet 为独立 Module），都需要重新运行此脚本确认推理结果不变。同时，测试脚本还可以导出中间 [tensor+](#)（如 [hidden_states+](#)、cos、sin、out_hidden_states 等）保存为 .npy 文件，作为后续 ncnn 模型验证的 reference 数据。

四、模型导出策略设计

4.1 分模块导出的必要性

Qwen3.5 是一个复杂的[多模态模型+](#)，不可能一次性将整个模型导出为单一的 ncnn 模型。原因包括：Vision Encoder 和 LLM Decoder 的输入输出维度差异极大；Decoder 的 RNN 结构不适合直接 trace；以及实际推理时 Vision Encoder 只执行一次而 Decoder 需要反复执行。因此，采用**分模块导出**策略，将模型拆分为若干独立的子模型分别导出。

整体导出清单如下：

导出顺序	模块名
------	-----

2	Vision Rotary Embedding	视觉 RoPE 的 $\cos/\sin$ 生成
3	Token Embedding	输入 token 到向量的映射
4	LM Head (out_proj)	最终输出 logits, 与 Token Embedding 共享权重
5	LLM Decoder	核心解码器, 包含全部 24 层

4.2 导出工具链

模型导出使用 **pnnx** (PyTorch Neural Network Exchange) 工具。pnnx 是 ncnn 生态的核心转换工具, 它接受 TorchScript 模型 (通过 `torch.jit.trace` 得到) 作为输入, 生成 ncnn 可加载的 `.param` / `.bin` 模型文件。pnnx 同时会生成一个 **Python** 文件 (`*_pnnx.py`), 该文件以 Python 代码的形式精确描述了导出的**网络拓扑**, 便于调试和验证。

导出脚本 `export_ncnn.py` 中, 每个子模型的导出流程大致为:

- 1. 从完整模型中提取目标子模块
- 2. 构造符合子模块输入规格的 dummy input
- 3. 调用 `torch.jit.trace` 生成 TorchScript 模型
- 4. 调用 pnnx 进行转换, 生成 ncnn 模型和 `pnnx.py` 文件

五、Vision Encoder 的导出与优化

5.1 初始导出

Qwen3.5 的 Vision Encoder 基于 **ViT 架构**, 原始模型中包含以下子模块: patch embedding (将图像切分为 patch 并做线性投影)、一系列 Transformer attention blocks、以及 merge patch 层 (将相邻视觉 token 合并以降低序列长度)。在初始导出方案中, 这些模块被分为独立的子模型分别导出。

5.2 合并 Merge Patch 到 Vision Encoder

单独导出 merge patch 模型意味着推理时需要多一次模型加载和执行, 增加了不必要的开销。观察到 merge patch 层本身计算量很小且与 Vision Encoder 的 attention blocks 紧密耦合, 因此修改 pnnx 导出的 `vision_encoder_pnnx.py`, **将 merge patch 的计算直接追加到 vision encoder 模型的尾部**。具体做法是在 `pnnx.py` 中手动编辑网络拓扑, 把 merge patch 的层定义和前向连接拼接到 vision encoder 的输出端。这样就成功将两个模型合并为一个, 减少了部署时的模型数量。

5.3 外部输入 RoPE 支持动态序列长度

Vision Encoder 中的 attention 层使用了 Rotary Position Embedding (RoPE)。原始导出时, RoPE 的 $\cos/\sin$ 值是在模型内部根据固定的序列长度计算的, 这意味着导出的模型只能处理特定长度的输入序列。为了支持不同尺寸的图像输入 (不同的**图像尺寸**意味着不同数量的 **visual token**, 进而对应不同的序列长度), 需要将 RoPE 从内部计算改为**从外部输入**。

修改 `vision_encoder_pnnx.py`, 将 RoPE 的 $\cos$ 和 $\sin$ tensor 从模型内部的常量计算改为模型的外部输入节点。这样, 推理时可以根据实际输入图像的尺寸动态计算 RoPE 值, 再作为输入传入 Vision Encoder 模型。

5.4 双 Shape 导出实现动态序列长度兼容

为了解决这个问题，修改 `vision_encoder_pnnx.py` 中的 `export_pnnx` 部分，使用两套不同的 `shape` 来执行转换。pnnx 会对比两套 `shape` 的推理结果，自动识别哪些维度是动态变化的，从而在 ncnn 模型中保留这些维度的动态性。

例如，一套 `shape` 使用序列长度 196（对应 14x14 的 `patch grid`），另一套使用序列长度 784（对应 28x28 的 `patch grid`）。pnnx 比较两次推理后，会正确地将序列长度维度标记为动态维度，而将 `hidden_size` 等固定维度标记为静态维度。

至此，Qwen3.5 的 Vision Encoder 部分的 ncnn 转换基本完成。与前代 Qwen3-VL 相比，由于 Qwen3.5 去掉了 **DeepStack Features 桥接层**，Vision Encoder 的输出端不再有额外的 `deepstack feature tensor` 需要处理，整体流程更为简洁。

六、LLM 部分的导出：Token Embedding 与 LM Head

6.1 Token Embedding 的导出

Token Embedding 层（`model.embed_tokens`）负责将输入的 token ID 映射为对应的 hidden state 向量。这是一个标准的 `nn.Embedding` 层，导出过程相对简单。构造 dummy input 为一组 token ID，trace 后通过 pnnx 转换即可。

6.2 LM Head 的导出

LM Head（代码中的 `lm_head`，也称 `out_proj`）是模型的最后一层线性映射，将 Decoder 输出的 hidden state 映射为 vocabulary 上的 logits 分布。在 Qwen3.5 中，**LM Head 与 Token Embedding 共享权重**（`tied_weights_keys = {"lm_head.weight": "model.embed_tokens.weight"}`）。这意味着两者实际是同一个权重矩阵的不同使用方式：Embedding 用于查表，LM Head 用于矩阵乘法。

导出时，将 LM Head 单独包装为一个可 trace 的模块，利用共享的权重矩阵完成导出。

七、LLM Decoder 的导出：核心挑战

LLM Decoder 是整个转换过程中最复杂、最困难的部分。24 层 Decoder 中混合了 Full Attention 和 Gated DeltaNet 两种架构，后者包含循环计算和复杂的状态管理，对 trace 导出构成了严峻挑战。

7.1 直接 Trace 的困境

如果直接对 Qwen3.5 的 Decoder 进行 `torch.jit.trace`，会遇到一个致命问题：**Gated DeltaNet 模块中的 RNN 循环会被完全展开**。

Gated DeltaNet 的核心计算包含一个逐时间步的循环（在 chunk 模式下是对每个 chunk 的循环，在 recurrent 模式下是对每个 token 的循环）。`torch.jit.trace` 的工作原理是记录一次前向推理过程中所有操作的执行轨迹，对于循环结构，它会将循环的每一次迭代作为独立的计算节点记录下来。假设输入序列长度为 512、chunk size 为 64，仅 chunk 循环就会展开 8 次迭代；如果使用逐 token 的 recurrent 模式，循环将展开 512 次。每次迭代内部又包含大量的矩阵运算，最终生成的计算图将包含数以万计的冗余节点，不仅模型文件体积膨胀，而且完全丧失了动态序列长度的能力。

7.2 改写 Gated DeltaNet 为独立 Module

解决上述困境的核心思路是：**将 Gated DeltaNet 和 ShortConv 改写为独立的 `torch.nn.Module`**，然后利用 pnnx 的 **moduleop 功能** 将它们作为不可展开的大算子 (opaque operator) 导出。

具体操作步骤如下：

第一步，修改 transformers 仓库中 Qwen3.5 的模型实现文件 `modeling_qwen3_5.py`（即 `modeling_qwen3_next.py`）。将原本内联在 `Qwen3NextGatedDeltaNet.forward()` 方法中的 `gated delta rule` 计算逻辑，抽取为一个独立的 `GatedDeltaRule(nn.Module)` 类：

```
class GatedDeltaRule(nn.Module):
    def __init__(self, num_k_heads, num_v_heads):
        super().__init__()
        self.num_k_heads = num_k_heads
        self.num_v_heads = num_v_heads

    def forward(self, A_log, dt_bias, b, a, query, key, value, initial_state):
        beta = torch.sigmoid(b)
        g = -A_log.float().exp() * F.softplus(a.float() + dt_bias)
        # ... gated delta rule 核心计算 ...
        return core_attn_out, last_recurrent_state
```

第二步，同样地将 ShortConv 的 `causal conv1d + SiLU` 激活逻辑抽取为独立的 `ShortConv(nn.Module)`：

```
class ShortConv(nn.Module):
    def forward(self, weight, mixed_qkv, initial_state):
        mixed_qkv = mixed_qkv.transpose(1, 2)
        groups = weight.size(0)
        padding = weight.size(2) - 1
        seq_len = mixed_qkv.size(2)
        mixed_qkv = F.silu(F.conv1d(mixed_qkv, weight, padding=padding, groups=groups))
        return mixed_qkv.transpose(1, 2)
```

第三步，在 `Qwen3NextGatedDeltaNet` 的 `__init__` 中实例化这两个独立模块，并在 `forward` 中调用它们替代原来的内联计算。

第四步，重新运行 `test.py` 验证改写后的推理结果与原始模型完全一致。这是不可省略的关键步骤——任何细微的数值差异都可能导致生成质量严重退化。

7.3 利用 pnnx moduleop 导出

完成代码改写后，使用 pnnx 导出 Decoder 模型时，通过 **moduleop** 参数指定 `GatedDeltaRule` 和 `ShortConv` 为自定义大算子：

```
pnnx model.pt moduleop=GatedDeltaRule,ShortConv
```

pnnx 在 trace 和转换过程中，遇到 `GatedDeltaRule` 和 `ShortConv` 模块时，不会深入展开其内部计算图，而是将它们整体保留为一个自定义算子节点。这样，无论输入序列长度如何变化，这两个算子在 ncnn 模型中始终只是一个节点，其内部的循环计算由自定义算子的实现来处理。

7.4 双 Shape 导出支持动态序列长度

与 Vision Encoder 类似，Decoder 模型同样需要支持动态序列长度。在 [prefill 阶段](#)⁺，输入序列可能包含数百个 token（prompt + visual tokens）；在 decode 阶段，每次只输入 1 个 token。修改 `decoder_pnnx.py` 的 `export_pnnx` 部分，使用两套不同的 shape（例如序列长度 11 和序列长度 7）来导出，让 pnnx 自动推断动态维度。

八、ncnn 自定义算子的实现

8.1 Python 端的验证桥接

pnnx 导出的 ncnn 模型中，`GatedDeltaRule` 和 `ShortConv` 是自定义算子，ncnn 运行时需要注册这些算子的具体实现。第一步是先用 Python 桥接 PyTorch 实现进行验证。

ncnn 的 Python binding 提供了 `register_custom_layer` 机制，允许用 Python 类来实现自定义层。将 PyTorch 中 `gated delta rule` 和 `short conv` 的计算逻辑原封不动地搬到 ncnn 自定义层的 `forward` 方法中：

```
class GatedDeltaRule(ncnn.Layer):
    def forward(self, bottom_blobs, top_blobs, opt):
        A_log = torch.from_numpy(np.array(bottom_blobs[0]))
        dt_bias = torch.from_numpy(np.array(bottom_blobs[1]))
        # ... 从 bottom_blobs 读取所有输入 ...
        # 调用 torch 实现的 gated delta rule
        core_attn_out, last_recurrent_state = torch_recurrent_gated_delta_rule(
            query, key, value, g, beta, initial_state, ...)
        # 将结果写回 top_blobs
        top_blobs[0].clone_from(ncnn.Mat(core_attn_out.numpy()))
        top_blobs[1].clone_from(ncnn.Mat(last_recurrent_state.numpy()))
        return 0
```

注册并加载 ncnn 模型后执行推理，将输出与之前保存的 PyTorch reference 结果对比，确认 ncnn 模型在自定义算子桥接下计算正确。

8.2 添加 Cache State 的输入输出

验证计算正确后，下一步需要为自定义算子增加 **cache state 的输入输出**。在自回归推理中：

对于 **GatedDeltaRule**，需要接受上一步的 recurrent state 作为 `initial_state` 输入，并在计算完成后输出更新后的 `last_recurrent_state`。首次推理时 `initial_state` 为空（ncnn 中对应 `ncnn.Mat()`），后续每步将前一步的输出 state 作为下一步的输入。

对于 **ShortConv**，需要接受上一步的 conv state 作为输入，并输出更新后的 conv state。Conv state 维护了最近 `kernel_size - 1` 个时间步的历史输入，在逐 token 生成时用于模拟[因果卷积](#)⁺的滑动窗口行为。

8.3 精度对齐的陷阱：必须使用真实数据

在将自定义算子从 PyTorch Python 实现迁移到 ncnn C++ 实现时，精度对齐是一个关键环节。这里有一个重要的经验教训：**Gated DeltaNet 的精度数据，不能使用随机生成的测试数据。**

输入时， g 值的分布可能与真实模型运行时的分布差异很大，导致验证通过但实际推理时出现 **NaN 爆炸**。因此，必须事先用 PyTorch 跑一遍真实推理，保存 `hidden_states`、`cos`、`sin` 等中间 tensor 为 `.npy` 文件，再用这些真实数据作为 ncnn 自定义算子的测试输入。

8.4 C++ 自定义算子实现

确认 Python 桥接的正确性后，需要用 ncnn 的 **C++ API** 重新实现 `GatedDeltaRule` 和 `ShortConv` 算子，以获得真正可部署的性能。C++ 实现需要严格对齐 Python 版本的数值行为，包括：

- `l2norm` 的 **epsilon** 处理方式
- **float32** 的中间精度计算（即使模型输入是 `fp16`，核心计算部分需要在 `float32` 下进行以避免精度损失）
- `exp` / `sigmoid` / `softplus` 等激活函数的数值行为
- 矩阵运算的维度变换和内存布局

验证方法是对同一组真实输入数据，分别用 Python 版和 C++ 版的自定义算子计算，比较输出的最大绝对误差（`max abs error`）和相对误差，确保在 `float32` 精度下误差在可接受范围内（通常要求 $< 1e-5$ ）。

九、Cache State 注入：add_kvcache / add_gdrcache / add_convcache

导出并验证完 Decoder ncnn 模型后，最后一个关键步骤是为模型注入 cache state 的输入输出节点。由于 pnnx 导出时模型内部并没有显式的 cache 管理逻辑（cache 在 HuggingFace 实现中由外部的 `DynamicCache` 类管理），我们需要**后处理** ncnn 模型文件，为每个需要 cache 的算子添加额外的 input/output blob。

这通过三个专门的脚本完成：

9.1 add_kvcache.py —— 为 SDPA 注入 KV Cache

该脚本处理 Decoder 中 6 个 Full Attention 层的 KV Cache。对于每个 Full Attention 层，在 ncnn 的 `.param` 文件中：

- 为 Scaled Dot-Product Attention (SDPA) 算子添加两个额外输入 blob：`cache_k{i}` 和 `cache_v{i}`，用于接收上一步的 key 和 value cache
- 添加两个额外输出 blob：更新后的 key 和 value cache，包含当前步拼接上历史 cache 的完整 KV 序列

这样，自回归推理时每一步只需输入当前 token 的 hidden state 以及上一步缓存的 KV 张量，SDPA 内部完成 cache 的拼接和注意力计算。

9.2 add_gdrcache.py —— 为 GatedDeltaRule 注入 Recurrent State Cache

该脚本处理 18 个 Linear Attention 层的 Gated DeltaNet recurrent state。对于每个 `GatedDeltaRule` 自定义算子：

- 添加一个额外输入 blob：`cache_gdr{i}`，用于接收上一步的 recurrent state（形状为 `[num_heads, k_dim, v_dim]`）
- 添加一个额外输出 blob：更新后的 recurrent state

9.3 add_convcache.py —— 为 ShortConv 注入 Conv State Cache

该脚本处理 18 个 Linear Attention 层的短卷积状态。对于每个 ShortConv 自定义算子：

- 添加一个额外输入 blob： `cache_conv{i}`，维护最近 `kernel_size - 1` 步的历史输入
- 添加一个额外输出 blob：更新后的 conv state

这三个脚本本质上都是对 ncnn .param 文件的**文本编辑操作**——修改算子的输入输出数量、添加新的 input/output blob name。操作完成后，ncnn 的 Decoder 模型就具备了完整的 cache 管理能力。

从最终的 ncnn 测试脚本中可以清晰看到这三类 cache 的使用方式：

```
# KV Cache (6 个 Full Attention 层)
ex.input("cache_k0", ncnn.Mat()) # 首次为空
ex.input("cache_v0", ncnn.Mat())
# ... cache_k1~k5, cache_v1~v5

# GDR Cache (18 个 Linear Attention 层)
ex.input("cache_gdr0", ncnn.Mat())
# ... cache_gdr1~gdr17

# Conv Cache (18 个 Linear Attention 层)
ex.input("cache_conv0", ncnn.Mat())
# ... cache_conv1~conv17
```

十、完整推理流程与验证

10.1 Prefill 阶段

Prefill 阶段处理完整的输入序列（包括系统 prompt、用户文本和可能的视觉 token）：

1. **图像编码**：将输入图像送入 Vision Encoder ncnn 模型，配合外部计算的 RoPE cos/sin，得到视觉 token 序列
2. **Token 嵌入**：将文本 token ID 通过 Token Embedding 模型映射为向量，与视觉 token 拼接
3. **Decoder 推理**：将拼接后的完整 hidden states 序列、[attention mask](#)⁺、RoPE cos/sin 送入 Decoder 模型，所有 cache 输入为空 `ncnn.Mat()`
4. **取 Logits**：将 Decoder 输出的最后一个位置的 hidden state 通过 LM Head 模型得到 logits
5. **保存 Cache**：从 Decoder 的输出中提取所有 cache 张量，供下一步使用

10.2 Decode 阶段（逐 Token 生成）

Decode 阶段每次只处理新生成的一个 token：

1. **Token 嵌入**：将上一步选出的 token ID 通过 Token Embedding 模型映射为向量（序列长度为 1）
2. **Decoder 推理**：输入长度为 1 的 hidden state、对应位置的 RoPE cos/sin，以及上一步保存的全部 cache 张量
3. **取 Logits**：通过 LM Head 得到下一个 token 的概率分
4. **更新 Cache**：用本步输出的 cache 替换上一步的 cache

10.3 结果验证

使用之前 PyTorch 导出的 reference 数据进行端到端验证。将相同的输入 `hidden_states` 、`cos` 、`sin` 送入 ncnn Decoder 模型，比较输出的 `out_hidden_states` 与 PyTorch reference 之间的误差。对于 float32 推理，期望最大绝对误差在 $1e-4$ 量级以下，均方误差[★]在 $1e-6$ 量级以下。

十一、关键经验与注意事项

11.1 关于模型拆分粒度

模型拆分的粒度需要权衡。太粗（例如整个 Decoder 一次导出）会遇到 trace 展开和动态形状的问题；太细（每一层单独导出）会导致推理时频繁的模型加载和上下文切换[★]开销。实践表明，按照“功能模块”拆分（Vision Encoder、Token Embedding、LM Head、LLM Decoder）是一个合理的平衡点。

11.2 关于 moduleop 的边界选择

选择哪些模块作为 pnnx moduleop 导出需要仔细考量。核心原则是：**包含循环、条件分支或其他 trace 不友好结构的模块应作为 moduleop**。在 Qwen3.5 中，GatedDeltaRule 的逐 chunk/逐 token 循环和 ShortConv 的因果卷积是明确的 moduleop 候选。而标准的 Linear、RMSNorm、SiLU 等层可以被 pnnx 正确转换为 ncnn 原生算子，无需额外处理。

11.3 关于精度与数值稳定性

Gated DeltaNet 的数值稳定性是整个转换过程中最容易踩坑的地方。门控衰减因子 g 的累积 $\exp$ 运算在 float16 下极易溢出，因此：

- 核心计算必须在 float32 精度下进行
- 精度对齐必须使用真实推理数据，不能使用随机数据
- 长序列测试时需要特别关注 NaN 和 Inf 的出现

11.4 关于 Qwen3.5 相对 Qwen3-VL 的架构简化

Qwen3.5 相较于 Qwen3-VL 去掉了 DeepStack Features 桥接层。这一改变直接简化了 Vision Encoder 的导出流程——输出端不再需要额外处理 deepstack feature tensor 的路由和合并逻辑。对于 ncnn 部署而言，这是一个值得注意的积极变化。

十二、总结

本文详细记录了将 Qwen3.5 0.8B 多模态大语言模型转换部署到 ncnn 框架的完整技术流程。核心工作量集中在以下几个方面：

阶段	核心工作	难度
Vision Encoder 导出	合并 merge patch、外部 RoPE、双 shape 动态维度	中

LLM Decoder 导出	改写 GatedDeltaNet/ShortConv 为独立 Module、pnnx moduleop 导出	高
自定义算子实现	Python 桥接验证、C++ 实 现、真实数据精度对齐	高
Cache State 注入	三类 cache 的 param 文件 后处理	中

Qwen3.5 的混合架构（Full Attention + Gated DeltaNet Linear Attention + Sparse MoE）代表了[大语言模型架构](#)⁺的演进方向——通过线性注意力降低长序列推理的计算复杂度，通过 MoE 在不显著增加推理开销的前提下扩大模型容量。成功将这种前沿架构部署到 ncnn，证明了 ncnn 生态在应对新型模型架构时的灵活性和[可扩展性](#)⁺，也为后续更大规模模型的[边缘部署](#)⁺积累了宝贵的工程经验。

Guided by nihui, written by Claude Opus 4.6

所属专栏 · 2026-06-26 16:55 更新



ncnn
 nihui
74 篇内容 · 6293 赞同

订阅

最热内容 · PNNX: PyTorch Neural Network Exchange

编辑于 2026-03-31 09:45 · 上海

[ncnn](#) [qwen3.5](#) [LLM](#)

想转行去做IC设计，有必要报网上的就业班嘛？

报班是为了买项目经验，买一套系统性的知识体系转行要么自学，要么报班但是自学，你根本没有项目，没有人带，甚至搭错环境...

爱说实话的云一哥 100+热议 ×



理性发言，友善互动

13 条评论

默认 最新



项明

这个方案支持高通的NPU算力计算吗，比方说8295等车机芯片有30tops算力可以使用
05-16 · 湖北

回复 喜欢



洋仔仔

从群里过来的
03-20 · 上海

回复 喜欢



风筝

看实现还是采用recurrent方式进行计算，期待更新一个chunk方式的实现 提升prefill性能
03-19 · 上海

回复 喜欢



影流之猪

tql
03-19 · 广东

...

...

...

...

03-18 · 江苏



如来来了

TQL

03-19 · 安徽

回复

喜欢

...

03-19 · 安徽



Molly

tql

03-18 · 上海

回复

喜欢

...

03-18 · 上海



映水之境

好耶~~

03-18 · 江苏

回复

喜欢

...

03-19 · 浙江



圈圈虫

tql，端侧推理框架唯二的光

03-19 · 浙江

回复

喜欢

...

03-19 · 浙江



圈圈虫 · 如来来了

MNN🤔

03-19 · 浙江

回复

2

...

03-19 · 安徽



如来来了

另外个是啥

03-19 · 安徽

回复

1

...

03-18 · 山东



风的彷徨

tql

03-18 · 山东

回复

喜欢

...

点击查看全部评论 >

推荐阅读



Qwen-VL多模态大模型的微调与部署

Mark·AI

多模态RAG技术：从语义抽取到VLM应用与规模化挑战

导读 本次分享聚焦于多模态 RAG 的实现路径与发展前景。核心议题涵盖五方面：1. 基于语义抽取的多模态 RAG 2. 基于 VLM 的多模态 RAG 3. 如何 Scale 基于 VLM 的多模态 RAG 4. 技术路线的...

DataFunTalk

多模态训练--数据加载

自然语言数据加载：以 ChnSentiCorp 数据集的加载为例，展示一个简单场景的自然语言加载并encode为tensor的流程。数据结构如下图所示： 利用 datasets的load_dataset函数加...

怀念普罗米修斯

Data-juicer+ AFlow:多模态数据处理新范式

导读 本次分享介绍了通义实验室开源项目 DataJuicer 的发展历程：从 SIGMOD 发表的 1.0 纯文本预训练版本，到 NeurIPS Spotlight 接收的 2.0 多模态增强版。主要内容包括以下几个部分：1. ...

DataFunTalk